

~ From Vibes to Data

Evaluating Agents on Your Real Work



Ville - Staff Engineer at Datadog SDLC & AI DevX

```
$ rm **/{AGENTS, CLAUDE}.md
```



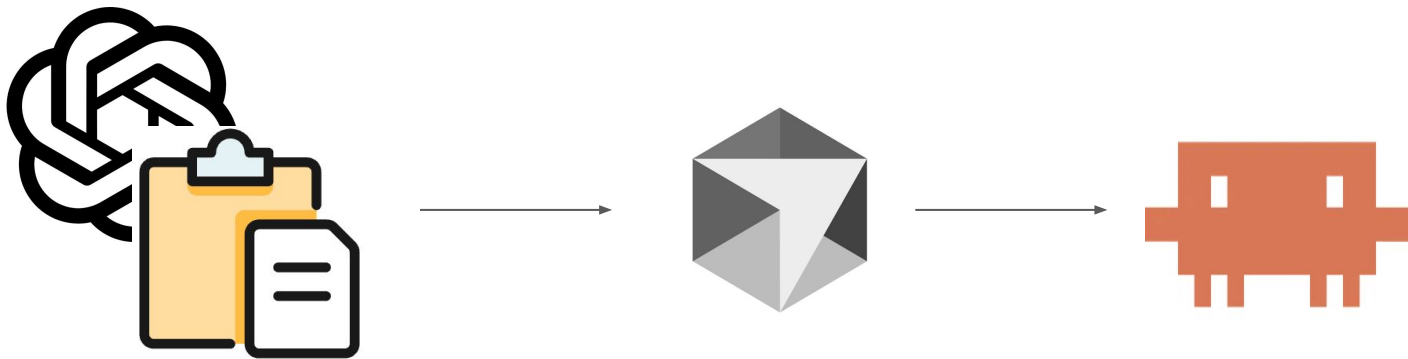
*one monorepo...YMMV...NFA...DYOR



Agent Performance
Token Cost



*one monorepo...YMMV...NFA...DYOR



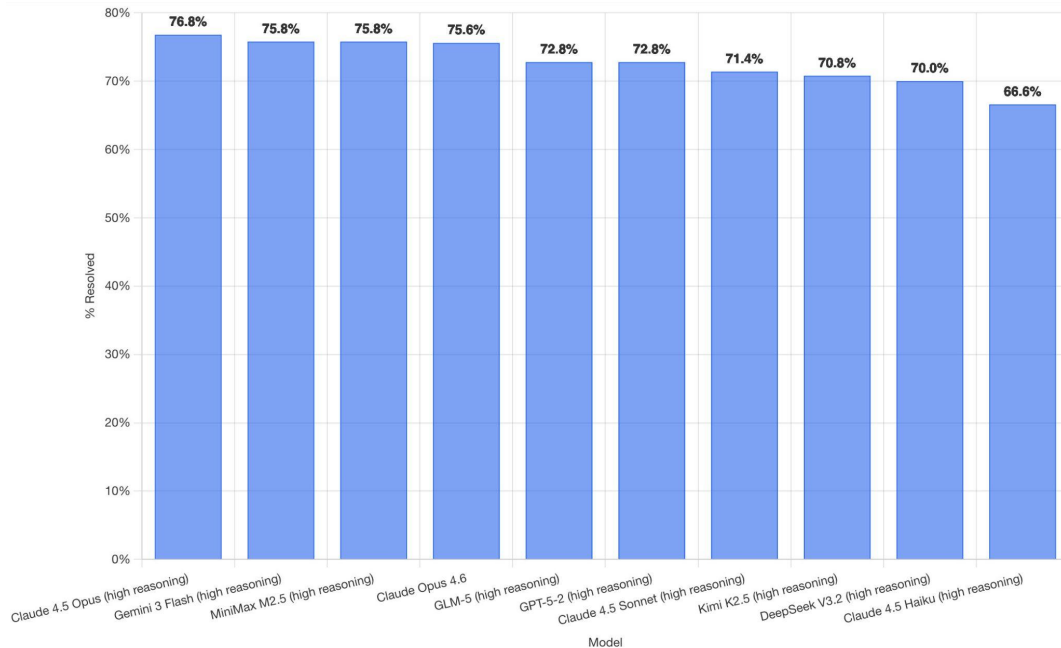
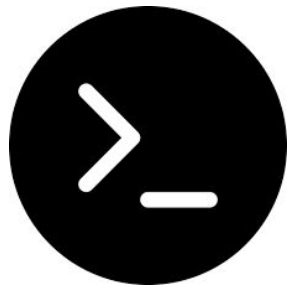
"We don't know how to make coding agents better inside Datadog"

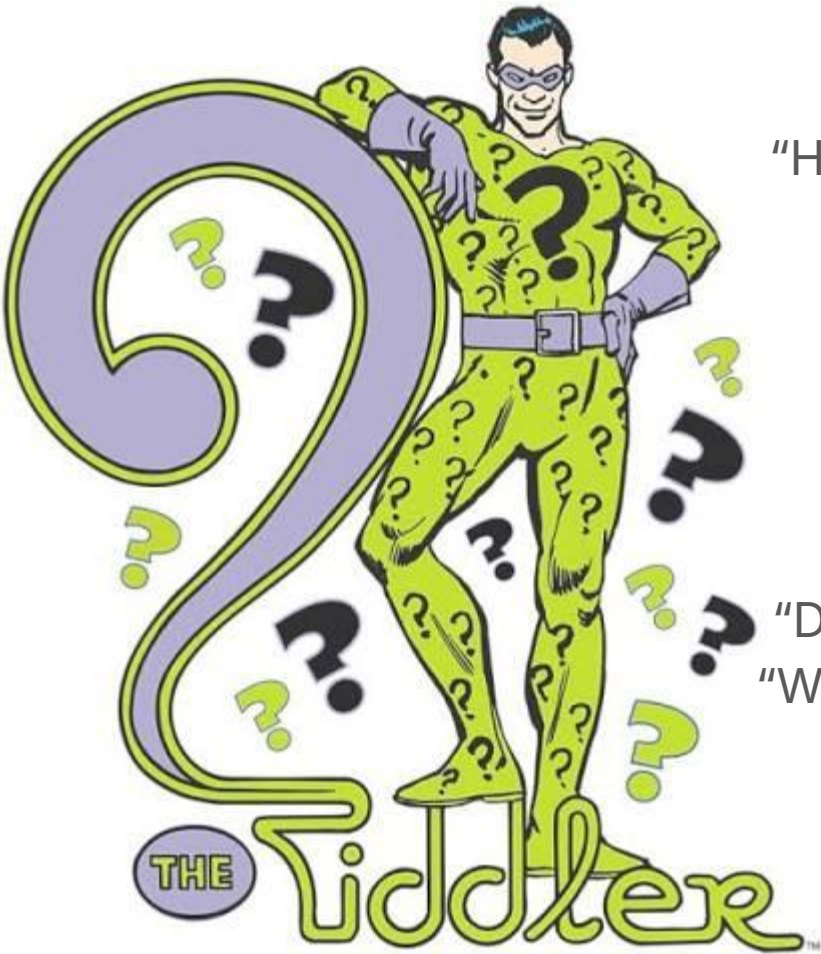
Public benchmarks answer something else

Swebench

Terminal Bench

Deep Swe





"How well do agents do at Datadog right now?"

"How do the models compare?"

"How much does the harness matter?"

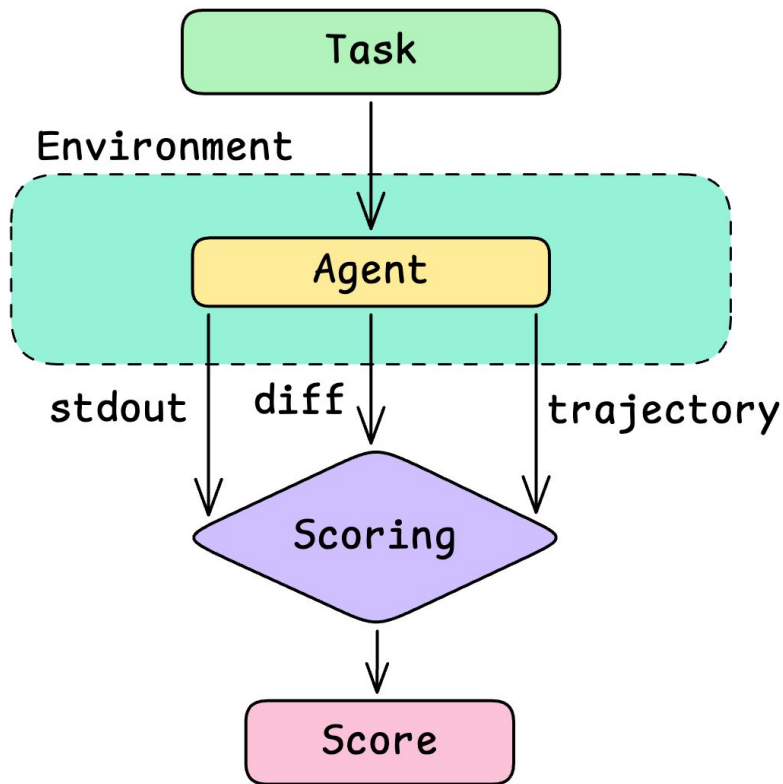
"How efficient is this agent vs that one?"

"Which MCP/Plugin/CLI works best here?"

"Does the new skill improve the performance?"

"What difference does the steering docs make?"

ADEEP - Agentic Datadog Engineering Evaluation Platform



Real work needs a realistic environment

Should be realistic

Access to source code

Similar config to what engineers have

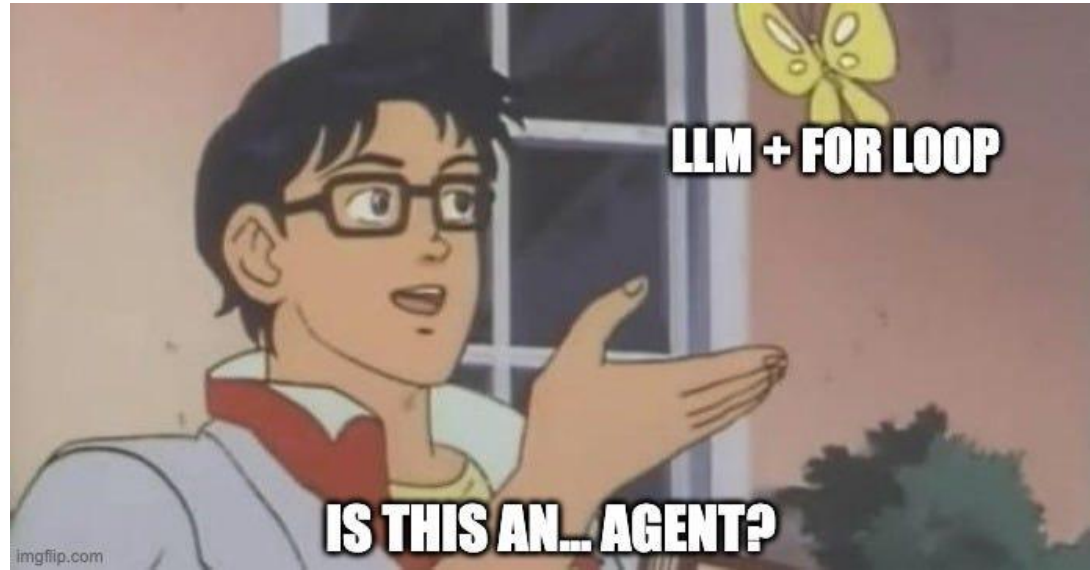
Clean environment



Tool + Model = Agent

Tool is the harness such as Claude Code, goose, OpenCode, pi

Model is the LLM like Opus 5, GPT 5.6 Luna, GLM 5.2, Devstral-24B



The Eval Task

"Create a new service for the examples domain.

It should be an HTTP API with a single endpoint that returns the requested number of fibonacci numbers.

For example, a GET request with argument 5 should return the first 5 fibonacci numbers: 1, 1, 2, 3, 5."



Eval anatomy & scoring

002-basic-service

- 0021-http-service
 - config.json
 - judge.md.tpl
 - prompt.md



Final score = (deterministic score * weight) + (llm judge score * weight)

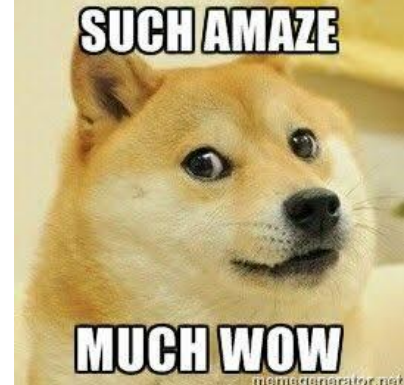
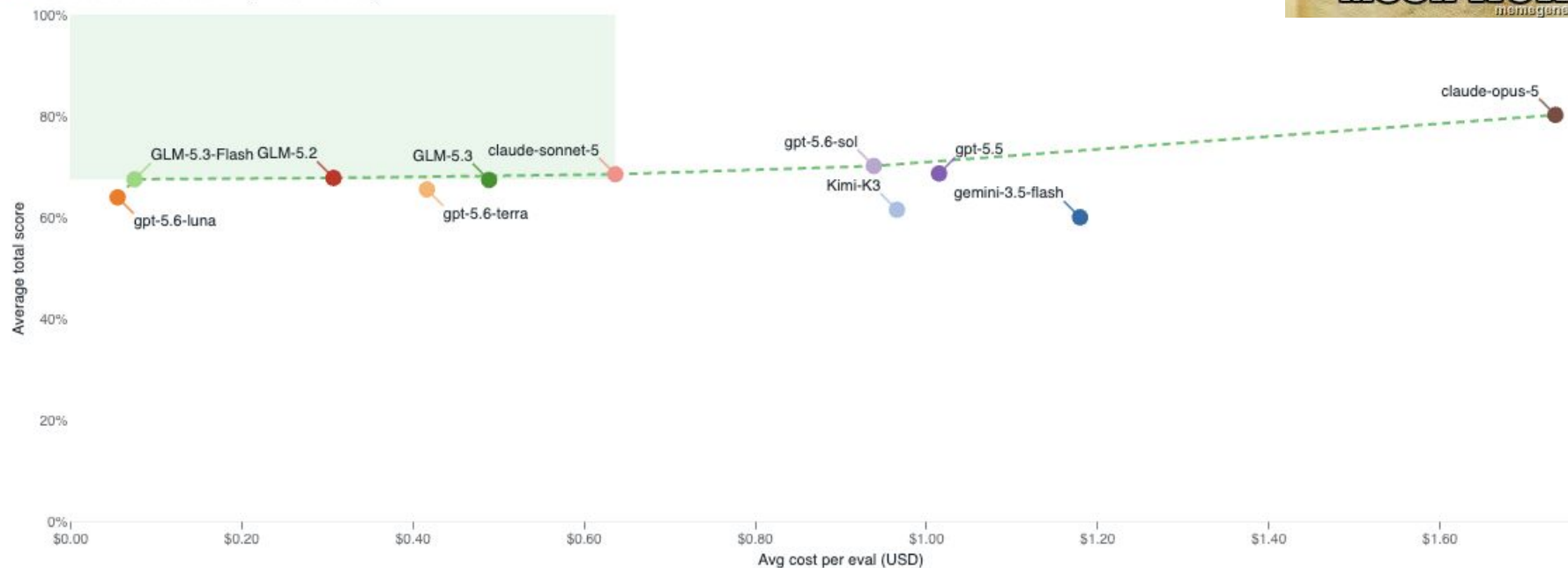
Nightly Baseline

Average score and cost by model

Each dot is a model aggregated over the selected environments, evals, and date range.

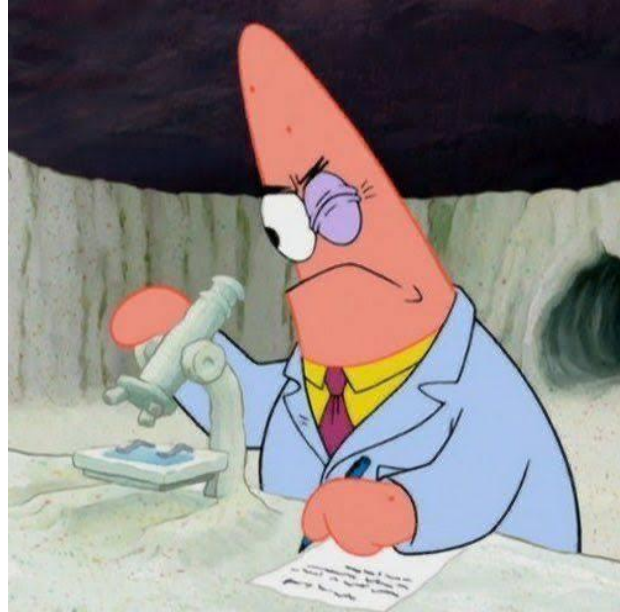
Below median cost, above median score

Best cost/score tradeoffs (Pareto frontier)



Experiments

- Caveman & RTK both increased token use by ~15%, Headroom worked (~25% improvement)
- Deleting all claude.md & agents.md files for our frontend monorepo improved agent performance
- A tested model router increases average cost & reduces scores
- Changing default model from Opus to Sonnet for Claude Code & Skills saved us ~\$650k/month



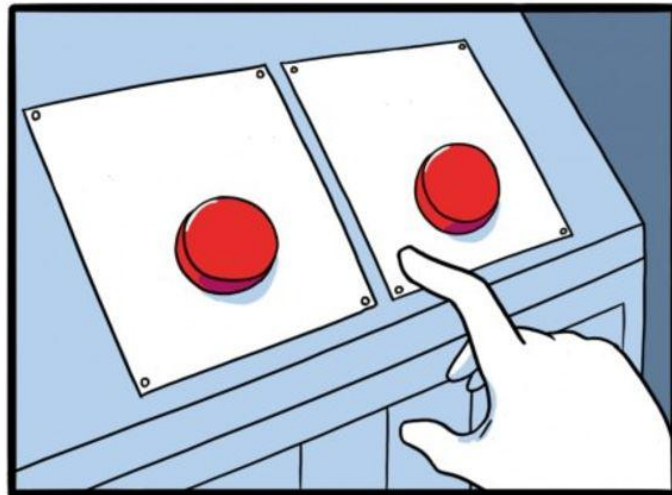
What's been hard/tricky

Test the task/workflow not the solution

Surprising results (sonnet outperforms opus)

Interpreting the results can be hard

Eval contamination & cheating



Context is an optimisation problem

Trading higher intelligence & more tokens to compensate for missing context

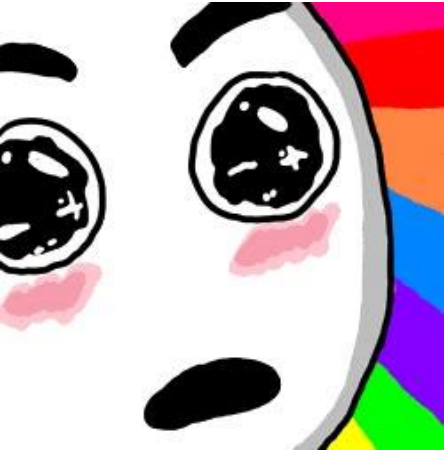
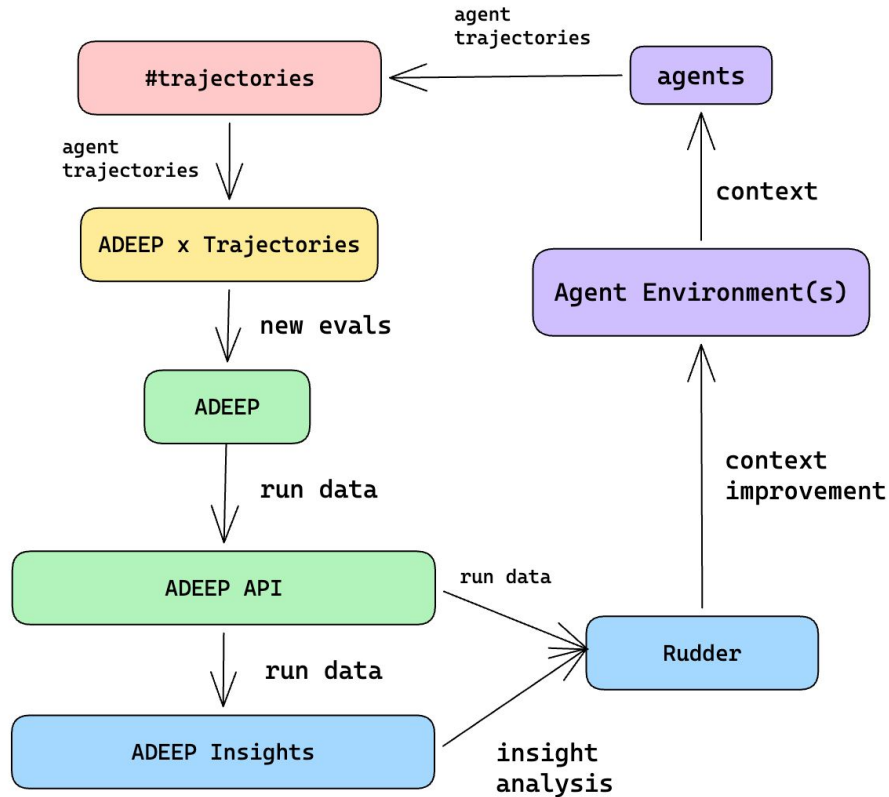
Lazy context strategy is additive, it should be an optimisation problem

Agent performance varies across repositories so you'll need your own benchmarks unless your software looks like open-source projects

Relatively small suite works well & the scores are remarkably stable



Organisational Learning Loop





@efexen



villehellman



THAT'S ALL, FOLKS!